

IPP 2025/2026 – Projekt: Interpret jazyka SOL26

Dokumentácia vylepšenej verzie

Ľuboš Noska (xnoskal00)
FIT VUT v Brne

22. apríla 2026

1 Celkový popis návrhu riešenia

Jadrom môjho riešenia je trieda **Evaluator** spolu s runtime triedami **SOLClass**, **SOLObject** a **Environment**. Tieto triedy nesú celú logiku behu programu – od vytvárania inštancií cez posielanie správ až po vyhľadávanie metód v reťazci dedičnosti. Trieda **Interpreter** (ktorej kostra bola súčasťou šablóny) nad nimi zastrešuje životný cyklus programu: registráciu užívateľských tried zo vstupu, statickú kontrolu arity metód a spustenie metódy `run` triedy **Main**.

Runtime model som navrhol minimalisticky. Každá hodnota v programe je **SOLObject**, každá trieda je **SOLClass** a každé volanie metódy alebo bloku vytvorí nové **Environment**. Pydantic uzly z AST som neprekladal do žiadnej druhej medzireprezentácie – **Evaluator** ich vyhodnocuje priamo. Vďaka tomu mám celý strom kedykoľvek k dispozícii (napríklad pri closure prostrediach blokov) a vyhol som sa duplicitu dvoch paralelných reprezentácií programu.

Posielanie správ je implementované jednou univerzálnou metódou `_eval_send`, ktorá postupne skúša tri cesty: (1) vyhľadanie užívateľskej metódy v reťazci dedičnosti, (2) vstavanú metódu podľa najbližšieho vstavaného predka a (3) prácu s inštančnými atribútmi (getter alebo setter podľa selektora a počtu argumentov). Ak ani jedna cesta neuspěje, vyhodí sa chyba 51. Dispatcher pre vstavané triedy (`_dispatch_builtin`) deleguje podľa typu na špecializované metódy (`_dispatch_string`, `_dispatch_integer`, `_dispatch_bool`, `_dispatch_nil`, `_dispatch_block`) s fallbackom na `_dispatch_object`. Toto rozdelenie podľa typu som zvolil, aby bol každý dispatcher krátky a aby pridanie novej vstavanej metódy nevyžadovalo zásah do hlavnej dispatch logiky.

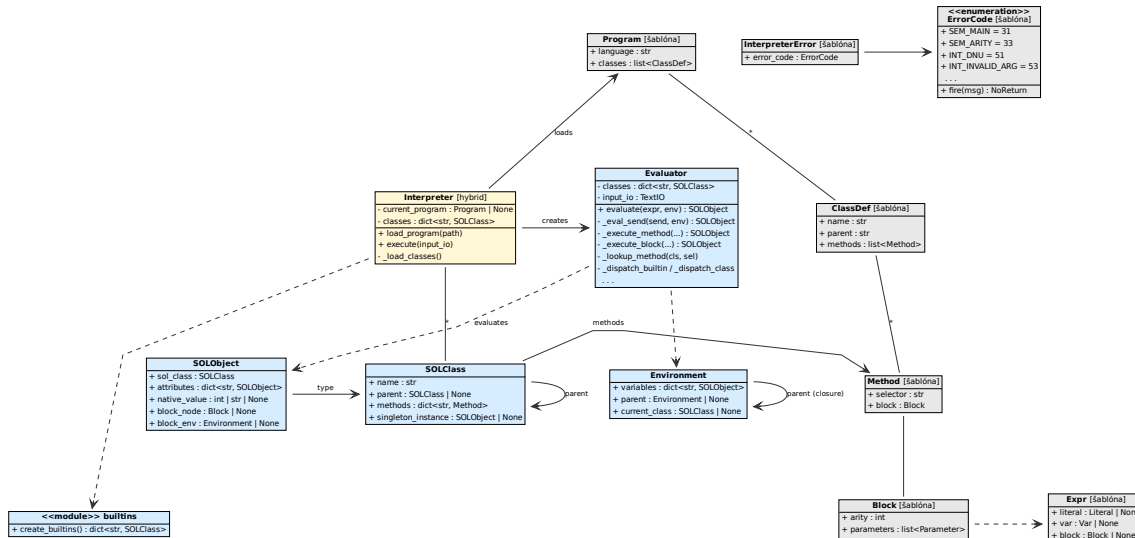
Najviac času som strávil nad rozlíšením `self`, `super` a bežnej referencie pri dedičnosti metód. V **Environment** si preto držím atribút `current_class`, ktorý označuje triedu, v ktorej je aktuálne bežiaci metóda *definovaná* (nie triedu objektu). `super` potom začína vyhľadávanie od rodiča tejto triedy, nie od triedy prijímateľa – inak by neprechádzali testy s viacúrovňovou dedičnosťou.

UML diagram tried je uvedený na obrázku 1. Triedy prevzaté zo šablóny sú odlíšené *šedou* farbou, trieda **Interpreter** (ktorej kostra bola súčasťou šablóny, ale hlavná logika je moja) je *svetlozltá* a moje vlastné runtime triedy a modul vstavaných tried sú *svetlomodré*.

2 Popis hlavných interných dátových štruktúr

SOLClass reprezentuje triedu počas behu programu. Obsahuje meno, odkaz na rodiča (`parent`), slovník metód (selektor $\rightarrow$ pydantic uzol `Method`) a voliteľne `singleton_instance` pre triedy `Nil`, `True` a `False`, u ktorých musí `new/from`: vždy vrátiť tú istú inštanciu.

SOLObject je jediná trieda pre každý objekt v behu programu – pre užívateľské inštancie, vsta-



Obr. 1: UML diagram tried interpretu SOL26. Šablónové triedy sú šedé, hybridná trieda **Interpreter** je žltá, moje triedy sú modré.

vané čísla, reťazce, booleany, `nil` aj bloky. Okrem odkazu na svoju triedu a slovníka `attributes` drží `native_value` (natívna Python hodnota pre `Integer` a `String`), dvojicu `block_node` a `block_env` (keď je objekt blokom – telo a zachytené closure prostredie) a dvojicu `is_class_ref` a `referred_class` (keď je objekt „literálom triedy“ ako `Integer`, ktorému možno poslať `new`).

`Environment` je premenné prostredie jedného volania metódy alebo bloku. Drží slovník lokálnych premenných, odkaz na rodičovský scope (pre closures blokov) a aktuálnu definičnú triedu metódy. Vyhľadávanie premennej prechádza `parent` reťazec až po najvrchnejšie prostredie, čo automaticky podporuje uzávery.

`Evaluator` je trieda bez stavu vo vzťahu k programu – vlastní len `classes` (register všetkých tried) a `input_io` (štandardný vstup). Všetky medziľahlé dáta počas behu putujú cez parametre, čo výrazne uľahčuje uvažovanie o rekurzii a closure sémantike.

3 Využitie návrhových vzorov a princípov OOP

Interpreter pattern. Celé jadro je priamočiara implementácia návrhového vzoru `Interpreter`. Uzly AST (`Expr` a jeho varianty) sú „gramatika“ a trieda `Evaluator` má jednu metódu `evaluate`, ktorá rekurzívne vyhodnocuje ľubovoľný uzol. Rozšírenie o nový typ výrazu by znamenalo pridať novú vetvu do `evaluate` a novú privátnu metódu `_eval_<typ>` – všetko ostatné zostáva nedotknuté.

Strategy / dispatch tables. Vyhodnocovanie správ vstavaných tried využíva niečo medzi tabuľkou funkcií a stratégiou: `_dispatch_builtin` podľa najbližšieho vstavaného predka delegovaním zavola jednu zo špecializovaných dispatcher metód (`_dispatch_string`, `_dispatch_integer`, ...). Tie vrátia `None`, ak daný selektor nepoznajú – `None` je potom signál, že sa má vyskúšať fallback na `Object`. Tento prístup mi dovolil pridávať nové vstavané metódy bez zmeny hlavnej dispatch logiky a zároveň podporuje dedičnosť od vstavaných tried (užívateľská `MyInt` : `Integer` automaticky „dedí“ všetky celočíselné operácie).

Composite. Objekt aj trieda majú spoločný polymorfný záujem na atribúte `parent` – v `SOLClass` ide o dedičnosť, v `Environment` o vnorenie scope. Obidve je možné prechádzať rovnakým

while-cyklom (`while current is not None: ...; current = current.parent`), čo robí pomocné metódy `_lookup_method`, `_builtin_base` a `_eval_var` veľmi krátkymi a podobnými.

Encapsulation a princíp jednej zodpovednosti. Každý súbor má jasnú úlohu – `error_codes` kódy chýb, `input_model` štruktúru vstupu, `sol_object` runtime reprezentáciu, `builtins` inicializáciu vstavaných tried, `interpreter` životný cyklus programu a `evaluator` samotné vyhodnotenie. Vďaka tomu sa po každej úprave jasne dá povedať, ktorého modulu sa zmena týka – napríklad pri oprave testov `INHERITANCE` som menil len `evaluator` a pri oprave `SEMERR_ARITY` len `interpreter`.

Singleton. Triedy `Nil`, `True` a `False` využívajú Singleton – `SOLClass` má nepovinný atribút `singleton_instance` a celá logika (`new`, `from`:, literál) sa cezeň zjednocuje do jedinej kontroly.

4 Popis najväčších problémov

Dispatch super v blokoch. Prvý problém, pri ktorom som narazil na strop, bolo, že keď je `super` použitý vo vnorenom bloku v metóde, pôvodný návrh (kde `current_class` bolo len v najvrchnejšom prostredí metódy) nevedel túto informáciu cez hranice blokov preniesť. Opravil som to metódou `_get_current_class`, ktorá prechádza `parent` reťaz prostredí a vráti najbližšie neprázdne `current_class`. Keďže uzávery bloku odkazujú na prostredie metódy, informácia sa nikdy nestratí.

Dedičnosť od vstavaných tried. Druhý problém súvisel s užívateľskou triedou, ktorá dedí priamo alebo nepriamo od `Integer`, `String` a pod. Keď takej inštancii program poslal `plus`: alebo `asString`, pôvodný dispatcher to nerozpoznal, lebo sa pozeral len na presné meno `sol_class.name`. Riešením je pomocná metóda `_builtin_base`, ktorá nájde najbližšieho *vstavaného* predka danej triedy. Dispatch potom funguje rovnako pre `Integer` aj pre užívateľskú `PositiveInt : Integer`.

Poradie dispatchu vs. metódy Object. Nakoniec – keďže každá užívateľská trieda implicitne dedí od `Object`, musel som rozhodnúť, v akom poradí sa skúšajú užívateľské a vstavané metódy. Správne poradie je: najprv užívateľské metódy v reťazi dedičnosti, potom vstavané metódy pre typ (ak sa jedná o vstavaný typ alebo jeho potomka), a nakoniec vstavané metódy `Object` (napríklad `asString`, `identicalTo`:). Inak by užívateľ nemohol prepísať `asString` v svojej triede.

5 Rozsah a dôvod úprav vo vylepšenej verzii

Pôvodné odovzdanie získalo v automatickom hodnotení 3,5/7,75 bodu. Najväčšie straty boli v kategóriách `SEMERR_ARITY` (0/40), `COMPLEX` (0/50), `INHERITANCE` (1/12), `CLASS` (2/9) a `CLOSURE` (3/14). Vylepšenia smerujú práve do týchto oblastí. Nižšie sú všetky zmeny chronologicky.

Interpret

- **Statická kontrola arity selektorov** (kategória `SEMERR_ARITY`). V `_load_classes` som doplnil porovnanie počtu dvojbodiek v selektore s počtom parametrov príslušného bloku a pri nezhode sa vyhadzuje chyba 33. Predtým sa nezhoda detekovala až dynamicky pri volaní, čím zlyhávali všetky testy, ktoré očakávali statickú detekciu.
- **Dedičnosť metód** (kategórie `INHERITANCE`, `CLASS`, `COMPLEX`). Metóda `_lookup_method` teraz vracia dvojicu (nájdenná metóda, trieda kde bola definovaná). Táto „definčná trieda“ putuje do `Environment.current_class`, vďaka čomu `super` hľadá v správnom bode hierarchie aj pri viacúrovňovej dedičnosti. Pribudli pomocné `_get_current_class` (získa informáciu cez `parent` reťaz prostredí, čím funguje aj z vnorených blokov) a `_builtin_base` (nájde najbližšieho vstavaného predka, čo umožňuje dedičnosť od `Integer`,

String a pod.). Upravil som aj poradie dispatchu v `_eval_send` – najprv užívateľské metódy v reťazi dedičnosti, až potom vstavané – aby ich bolo možné prepísať.

- **Konštruktor from:** (kategória `CLASS`, `RTERR_ARG`). Do `_dispatch_class` pribudla kontrola typu argumentu cez `_is_instance_of`. Ak argument nie je inštanciou prijímateľa ani jeho potomka, vyhadzuje sa chyba 53.
- **Singletony** `Nil/True/False` (kategória `ADVANCED`). Opravil som dispatchery tak, aby `new` a `from:` na týchto triedach vždy vracali tú istú inštanciu uloženú v `SOLC-lass.singleton_instance`. Zároveň som doladil vstavané predikáty `isNumber`, `isString`, `isBlock`, `isNil` a `isBoolean`, ktoré nevracali správnu hodnotu pre potomkov vstavaných tried.
- **Triedna metóda** `String read` a `Block new.read` bol predtým ošetrovaný len ako inštančná metóda – presunul som ho do `_dispatch_class` tak, aby sa dal volať ako `String read`. Podobne `Block new` teraz vytvára korektný prázdny blok s aritý 0, nie obyčajný `Object`.

Kontajnerizácia

Predvolený -interpreter v test stage. Pôvodne sa tester spúšťal bez argumentov, čo od používateľa vyžadovalo, aby vždy odovzdal `-interpreter`. Nový `ENTRYPOINT` má tento argument prednastavený na `python3 /src/int/src/solint.py`, takže priamo ukazuje na interpret v rovnakom obraze. Užívateľ ho môže kedykoľvek prepísať vlastným `-interpreter`, keďže ďalšie CLI argumenty sa štandardne pridávajú za hodnoty z `ENTRYPOINT`.

Wrappery pre nástroje kvality kódu. Aktualizovaný skript `is_archive_ok.sh` testuje dostupnosť nástrojov kvality (`ruff`, `mypy`, `eslint`, `prettier`) tak, že nad `/src/int/` a `/src/tester/` v kontajneri namontuje obsah archívu. Pôvodné riešenie vytváralo symlinky na tieto nástroje až počas `docker build` – mount ich potom prekryl a nástroje prestali byť viditeľné. Preto som do `int/` a `tester/` pridal malé shell wrappery `ruff`, `mypy`, `eslint`, `prettier`, ktoré sú súčasťou archívu a prežijú mount. V `check stage` som navyše pridal `npm install -g eslint prettier`, aby boli globálne dostupné bez ohľadu na to, či je `/src/tester/node_modules/` po mount-e dostupný. Samotný build-time lint som prepol na priame volanie `./node_modules/.bin/eslint` a `./node_modules/.bin/prettier`, aby používal verzie zamknuté v `package-lock.json`.

Testovací nástroj a dokumentácia

V testovacom nástroji som nerobil funkčné zmeny – úpravy sa týkali výlučne interpretu a Dockerfile. Samotná dokumentácia bola aktualizovaná o túto sekciu a o popis nových pomocných metód (`_builtin_base`, `_get_current_class`, `_is_instance_of`) v sekciách 1 a 4.

6 Popis využitia prostriedkov AI v projekte

V projekte som využíval chatbot **Gemini** (Google) ako doplnkovú pomôcku pri porozumení zadania a vlastného kódu. Konkrétne som ho používal na:

- **Vysvetľovanie šablóny projektu** – na začiatku som si dal postupne vysvetliť jednotlivé súbory šablóny (napr. `input_model.py`, `solint.py`, `error_codes.py`), aby som sa rýchlejšie zorientoval v tom, čo je už pripravené a kde budem dopĺňať vlastnú logiku.
- **Orientáciu v zadaní a špecifikácii SOL26** – pýtal som sa na význam jednotlivých sekcií zadania, formát `.test` súborov, pravidiel hodnotenia a štruktúru SOL-XML. Využil som to hlavne vtedy, keď som si nebol istý interpretáciou niektorej pasáže špecifikácie.
- **Písanie SOL-XML testovacích súborov** – nechal som si generovať vstupné XML scenáre (napr. `test_class*.xml`, `test_closure*.xml`, `test_singletons.xml`), na ktorých

som si priebežne overoval, či interpret funguje správne a či novo implementovaná funkcionálnosť neporušila skôr prejdene testy.

- **Písanie docstringov** – pri hotových moduloch som ho požiadal o vygenerovanie docstringov pre triedy a metódy.
- **Diskusia nad návrhom** – napríklad pri rozhodovaní, či singletonovú logiku riešiť v `SOLCClass` alebo v dispatcheri, alebo prečo musí `_lookup_method` vracať aj triedu, v ktorej bola metóda definovaná.

Screenshots z týchto konverzácií sú priložené v adresári `ai/` v koreni archívu.

7 Možnosti ďalšieho rozšírenia

Zo zoznamu rozšírení som si vybral dve, ktoré by bolo možné zapracovať s malým zásahom do existujúceho návrhu.

Podpora mechanizmu `doesNotUnderstand`

Aktuálne v `_eval_send` končia všetky neúspešné dispatch pokusy chybou 51. Rozšírenie by znamenalo zmeniť len tento jeden bod: namiesto chyby by sa na príjemcovi skúsilo poslať `doesNotUnderstand`: s argumentom typu `Message`. `Message` by bola nová vstavaná trieda s atribútmi `selector` a `args` a vstavanými metódami `selector`, `argumentAt`: a `argumentCount`. Registroval by som ju v `create_builtins` a pridal dispatcher `_dispatch_message`. Predvolenú implementáciu `doesNotUnderstand`: by poskytol `_dispatch_object` – vyhodil by pôvodnú chybu 51. Proti nekonečnej rekurzii stačí na `Evaluator` držať množinu už prebiehajúcich DNU dvojíc (príjemca, selektor).

Podpora rozhraní (interface)

V pydantic modeli by pribudla nová trieda `InterfaceDef` s atribútmi `name`, `extends` (zoznam nadradených rozhraní) a `selectors` (povinné metódy); `ClassDef` by dostal voliteľný atribút `implements`. Novú runtime štruktúru nepotrebujem – stačí v `Interpreter` viesť druhý register `interfaces`. Kontrolu urobím staticky v `_load_classes`: pre každú triedu s `implements` overím, že každý selektor rozhrania (vrátane zdedených) je nájdniteľný cez existujúce `_lookup_method`, a pri nesplnení vyhodím chybu 35. Zákaz inštanciacie rozhrania pridám jednou vetvou v `_dispatch_class`.